程式設計｜ Week 2

class 與運算子多載

讓自己的類別可以安全地保存資料，也能使用 ==、+、[]。

對照老師講義：第 11 章 Classes、第 12 章 Operator Overloading。

今天要完成什麼？

最後的作品：Vec2

`Vec2(2, 5)` 代表 x = 2、y = 5。你會逐步決定它如何建立、比較、相加和讀寫座標。

1. 先用 `class` 決定哪些資料能被外部直接碰到。
2. 再用成員函式定義物件能做的事。
3. 最後用運算子多載，讓 `a == b`、`a + b`、`a[0]` 有清楚的意思。

上課節奏：先猜結果 → 看幾行示範 → 自己寫 → 編譯執行。

第 1 章｜為什麼要把資料設為 private？

想像帳戶的餘額可以直接從外面改：

```
struct BankAccount {
    int balance;
};
BankAccount a{100};
a.balance = -500;
```

程式可以編譯，但餘額變成不合理的 -500。

改成 class 後

把 `balance` 放在 `private`，外面只能呼叫公開方法。公開方法可以先檢查輸入，再決定是否改餘額。

第 1 章｜建立物件時給初始值

```
class BankAccount {
private:
    int balance;
public:
    BankAccount() : balance(0) {}
    bool deposit(int amount) {
        if (amount <= 0) return false;
        balance += amount;
        return true;
    }
    int getBalance() const { return balance; }
};
```

`BankAccount()` 是**建構子**：建立物件時自動執行，讓餘額先成為 0。`deposit(50)` 回傳真且餘額變 50；`deposit(-3)` 回傳假，餘額保持 50。

1. **資料在哪裡？** `balance` 在每個帳戶物件裡。
2. **誰能改資料？** 類別內的方法可以；類別外不能直接寫 `a.balance = ...`。
3. **輸入是什麼？** `deposit` 收到一個整數 `amount`。
4. **結果是什麼？** 正數才增加餘額，並回傳是否成功。

換成自己的題目

`Vec2` 要保存兩個座標。建立它時應給什麼值？外面需要直接改 `x`、`y` 嗎？

第 1 章完成

接下來可以練習第 1 關

從空白檔定義 Vec2，自己寫建構子、讀取與印出。

打開 week2 ／學生練習.md，先預測，再自己打程式與測試。

第 2 章｜同一個物件，有些方法會改它

```
class Counter {
private:
    int count;
public:
    Counter(int start) : count(start) {}
    void addOne() { count += 1; }
    int value() const { return count; }
};
```

`addOne()` 把這個 Counter 的 `count` 加 1。`value()` 只讀取並回傳 count；方法後面的 `const` 表示它不修改這個物件。

第 2 章｜ const 物件只能呼叫不修改它的方法

```
const Counter frozen(5);
cout << frozen.value();  // 5
// frozen.addOne();       // 編譯錯誤
```

`frozen` 建立後不允許改值。呼叫 `value()` 沒有問題；`addOne()` 會改值，所以不能呼叫。

準備寫 Vec2

`move(dx,dy)` 會改座標；`lengthSquared()` 只計算 $x^2 + y^2$，不改座標。哪一個方法後面需要 `const`？

第 2 章完成

接下來可以練習第 2 關

讓 `Vec2` 可以移動，並計算移動後的長度平方。

打開 week2 ／學生練習.md，先預測，再自己打程式與測試。

第 3 章｜ a == b 其實是在呼叫函式

```
class Box {
private:
    int value;
public:
    Box(int v) : value(v) {}
    bool operator==(const Box& other) const {
        return value == other.value;
    }
};
```

`a == b` 可以讀成 `a.operator==(b)`：左邊是呼叫函式的物件，右邊傳入 `other`。若 a 的值是 5、b 的值也是 5，結果是真；若 b 的值是 8，結果是假。

```
bool operator==(const Box& other) const
```

- `const Box& other`：用右邊原本的物件，且不修改它。
- 最後的 `const`：這個方法不修改左邊物件。
- 回傳 `bool`：比較結果只有真或假。

!= 可以怎麼寫？

如果已經會判斷「相等」,「不相等」可以直接使用它的相反結果；不必再複製一次比較條件。

第 3 章完成

接下來可以練習第 3 關

讓兩個 Vec2 能用 == 和 != 比較。

打開 week2 ／學生練習.md，先預測，再自己打程式與測試。

設 a = (1,2)、b = (3,4)。執行 a += b 後：

物件	執行前	執行後
a	(1,2)	**(4,6)**
b	(3,4)	(3,4)

只有左邊 a 改變。右邊 b 只是提供要加進來的值。

```
Counter& operator+=(const Counter& other) {
    count += other.count;
    return *this;
}
```

`this` 指向正在執行方法的左邊物件；`*this` 就是左邊物件本身。

回傳參考的用途

`(a += b) += c` 要在第一次加完後，繼續修改同一個 *a*。因此回傳 `Counter&`，而不是另外造一份 Counter。

第 4 章｜自己檢查兩種容易漏的情況

1. `a += b` 之後，`b` 應保持原樣。
2. `a += a` 時，左右其實是同一個物件；每個座標應變成原來的兩倍。

寫之前先說出來

`Vec2` 有 `x`、`y` 兩個欄位。若把 `other.x` 加到 `x`，下一行應把哪個值加到 `y`？

第 4 章完成

接下來可以練習第 4 關

寫 +=，確認它只改左邊，並能連續使用。

打開 week2 ／學生練習.md，先預測，再自己打程式與測試。

第 5 章｜ + 和 += 的差別

一樣設 a = (1,2)、b = (3,4)：

寫法	得到的結果	原本的 a
`a += b`	a 變 (4,6)	改變
`c = a + b`	新的 c 是 (4,6)	保持 (1,2)

+ 要產生新物件。如果它偷偷改掉 a，後面的程式會很難預測。

第 5 章｜先複製，再重用 +=

```
Counter operator+(const Counter& other) const {
    Counter result(*this);
    result += other;
    return result;
}
```

`result` 是新物件；只改它，不改左右原物件。`return result;` 回傳值。

為什麼不能回傳 Counter&？

`result` 是函式內的區域物件。函式結束後它就不存在；若回傳它的參考，呼叫端會拿到無效的參考。

第 5 章完成

接下來可以練習第 5 關

寫 +，產生新 Vec2，保留左右原值。

打開 week2 ／學生練習.md，先預測，再自己打程式與測試。

第 6 章｜回傳「值」和「參考」有何不同？

```
int x = 2;
int copy = x;
copy = 9;       // x 仍是 2

int& alias = x;
alias = 9;      // x 變成 9
```

若 `v[0]` 只回傳一個整數值，`v[0] = 9` 無法改到原本的 `x`；可寫入版本需要回傳欄位的**參考**。

第 6 章｜同一個 []，兩種使用方式

```
int  operator[](int index) const; // const 物件：讀取
int& operator[](int index);       // 一般物件：可改欄位
```

- `const Vec2 fixed(3,4); fixed[1]`：讀取 y，得到 4。
- `Vec2 v(1,2); v[0] = 9;`：改寫 x，v 變 (9,2)。
- `fixed[1] = 9;` 不能編譯，因為 fixed 是 const 物件。

索引邊界

我們只約定 0 是 x、1 是 y；其他索引要明確報錯。錯誤索引會結束程式，請另開一次執行來測。

第 6 章完成

接下來可以練習第 6 關

寫兩個 [] ，讓一般物件可改、const 物件只讀。

打開 week2 ／學生練習.md，先預測，再自己打程式與測試。

第 7 章｜左邊是誰，決定能呼叫誰的方法

設 v = (2,5)，我們約定「加整數」是兩個座標都加上它。

- `v + 3`：左邊是 `Vec2`，可以呼叫 `Vec2` 的成員運算子。
- `3 + v`：左邊是 `int`，不會去呼叫 `Vec2` 的成員運算子。

兩個式子都應產生 (5,8)，而 v 仍保持 (2,5)。第二種寫法需要**類別外的函式**。

第 7 章｜用 Box 看兩種位置

```
// Box 類別內：box + 3
Box operator+(int n) const {
    return Box(value + n);
}

// Box 類別外：3 + box
Box operator+(int n, const Box& box) {
    return box + n;
}
```

類別外版本重用 box + n，不需要碰 Box 的私有欄位。

注意

Box 只有一個值；你要寫的 Vec2 有兩個座標。兩個座標都要加上整數。

第 7 章｜下筆前的最後整理

運算	左邊物件會改嗎？	主要回傳什麼？
`a == b`	不會	真／假
`a += b`	會	左邊物件的參考
`a + b`	不會	新物件
`v[0] = 9`	會	欄位的參考
`v + 3`	不會	新物件

先決定行為，再決定函式的參數、const 與回傳型別。

第 7 章完成

接下來可以練習第 7 關

讓 v + 3 和 3 + v 都得到新向量。

打開 week2 ／學生練習.md，先預測，再自己打程式與測試。